

GAMESCRAFTERS · ROBOTICS

Rebuilding GamesmanROS on ROS 2

An architecture rewrite that replaces a blocking monolith with a lifecycle-managed node graph — fixing the motion planning, gripper, and recovery failures that made the original system unreliable.

ROS 2 Humble · MoveIt 2 / OMPL · OpenCV · MyCobot 280 Pi
Status: 7 nodes implemented · pre-hardware-validation

Three failures that weren't bugs — they were architecture

FAILURE 01

Motion planning

`set_planner_id("LIN")` forced every move onto a straight Cartesian line. On the MyCobot's constrained workspace, that line routinely passed through IK-infeasible configurations.

Result: planning returned `False`, the move was silently skipped, and the game continued as if the piece had moved.

FAILURE 02

Gripper control

The gripper topic was fire-and-forget with no delivery guarantee, so the workaround was to publish the same command 7+ times with `sleep()` between each.

Result: ~3s lost per actuation, and no confirmation the gripper ever actually closed before the arm moved on.

FAILURE 03

No lifecycle or recovery

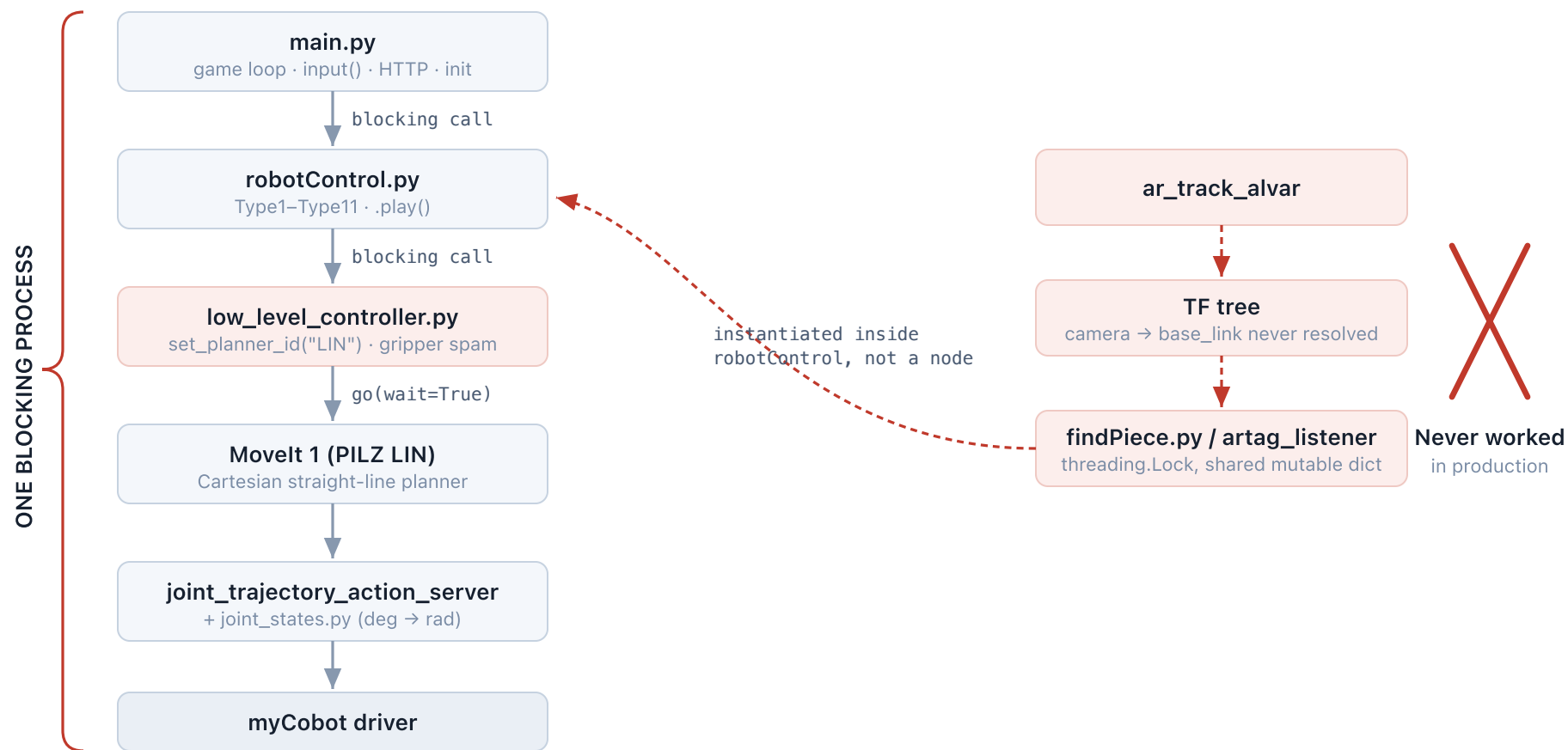
`main.py` ran the game loop, node setup, user input, and HTTP calls in one blocking script. Vision was instantiated *inside* the controller behind a raw `rospy.spin` thread.

Result: any failure anywhere required killing and restarting the entire process.

The common root cause: the system was a Python call stack that happened to use ROS, not a distributed system. Every fix below is structural, not a patch.

BEFORE

ROS 1: one synchronous chain



Consequences

No failure signal reaches the caller — plans that fail return False and are dropped.

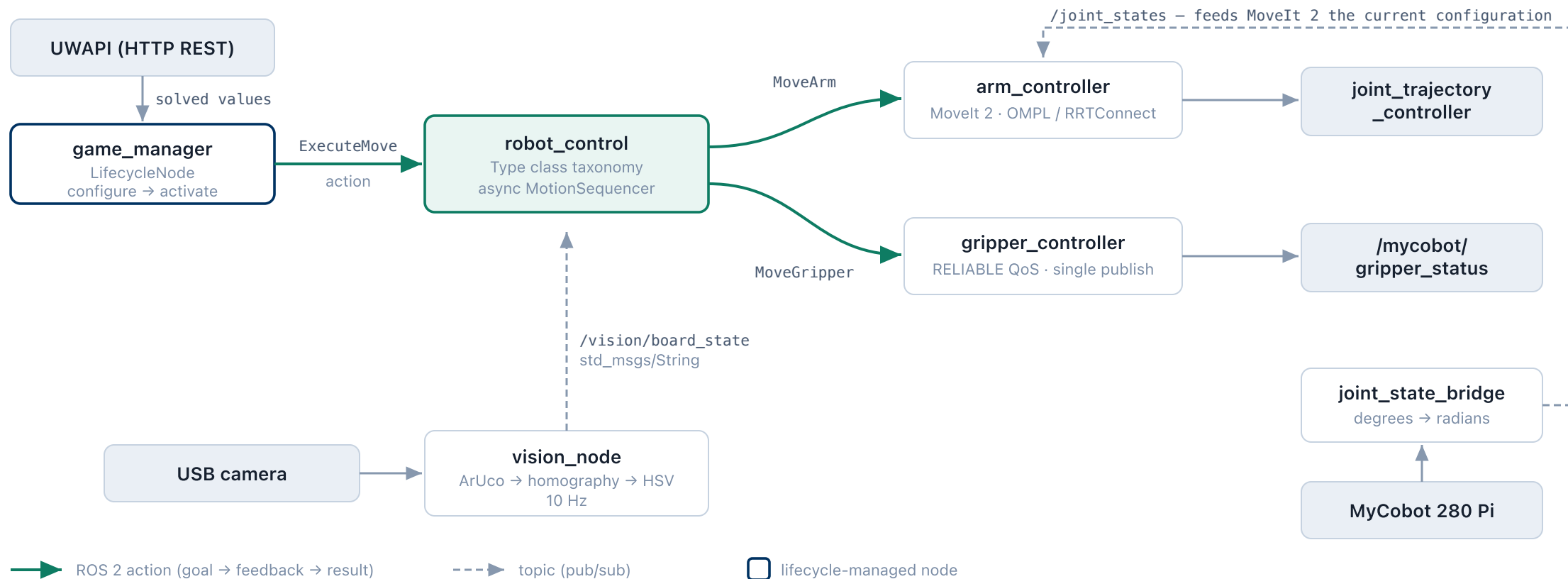
No node can be restarted independently of any other.

Vision state is shared memory across threads, not a message.

Swapping the planner or the camera means editing the monolith itself.

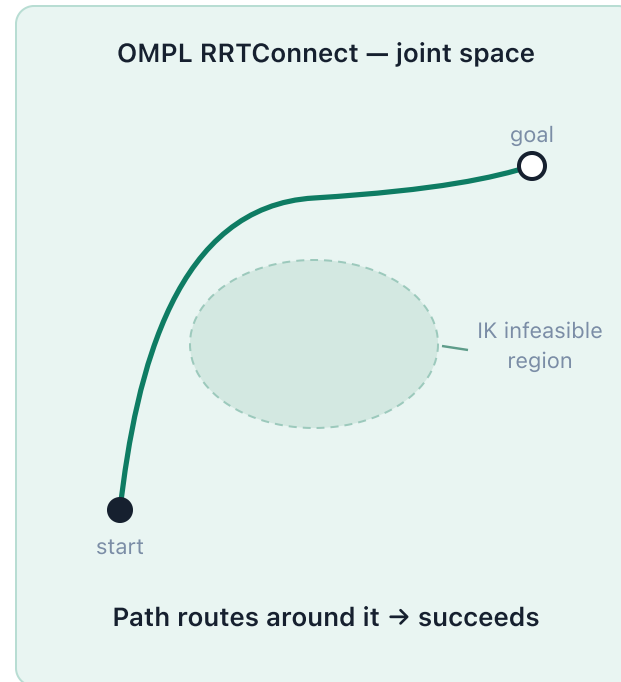
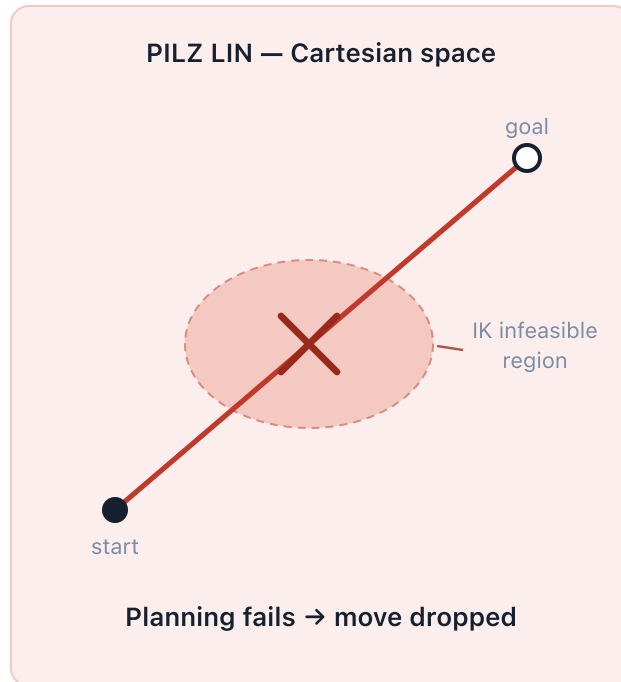
AFTER

ROS 2: a graph of independent nodes



Every node restarts independently · every hop is typed, non-blocking, and reports failure back to its caller

Cartesian straight lines → joint-space search



Why LIN failed

LIN interpolates the end-effector linearly through 3D space. MoveIt must solve IK at every *intermediate waypoint*. One infeasible point anywhere along the line kills the whole plan.

Why OMPL works

RRTConnect searches the 6-DOF joint configuration space directly, growing trees from both start and goal. The Cartesian path is unconstrained, so it can route around singularities and joint limits.

The trade-off we accepted

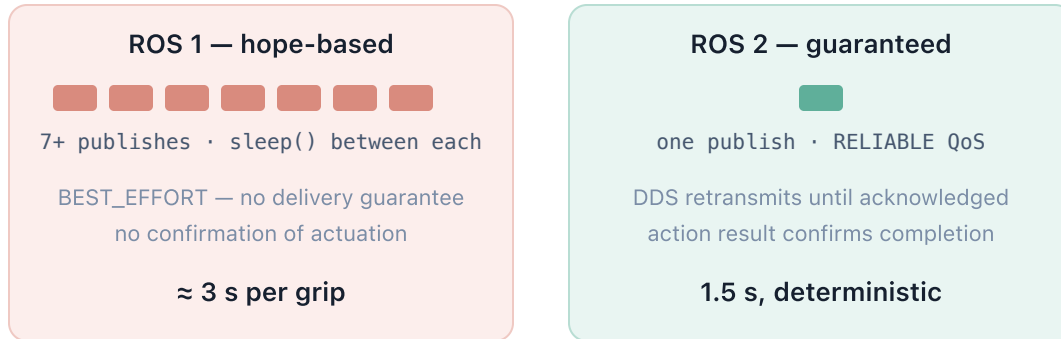
Motion no longer looks like a clean straight line to a human observer. For pick-and-place on a game board, reliability beats aesthetics.

IK is solved only at the two endpoints, never at interpolated waypoints in between

This is a one-line change in `arm_controller.py` — but it is the change the whole rewrite was for. Every other fix exists to make sure that when a plan does fail, the system finds out about it.

Guaranteed delivery, and a system that can recover

GRIPPER RELIABILITY



The caller `await` s the action result, so the arm physically cannot begin the next motion before the gripper has finished actuating.

LIFECYCLE & RECOVERY



In ROS 1 a mid-game failure meant killing the process. Here `game_manager` can be deactivated, reconfigured with a new game, and reactivated without the motion stack ever restarting.

Both fixes share a shape: the original code guessed that something had happened and moved on. The rewrite makes the system wait for confirmation — from DDS for delivery, from the action result for completion, and from the lifecycle manager for readiness.

Dropping the TF chain for a homography



What broke in ROS 1

`ar_track_alvar` needed accurate camera intrinsics, published poses in the camera frame, and depended on a camera → `base_link` TF transform that was never reliably established. Three stacked dependencies, each a failure point.

Why homography removes all three

Four markers at known board corners fully determine the projective transform. Camera tilt, height and angle are absorbed by the warp — no intrinsics file, no TF tree, no extrinsic calibration step.

Why HSV, not a learned model

Pieces are two solid colors on a flat board. Hue is lighting-invariant, detection is deterministic, it runs at frame rate on a Pi, and there is no training set to collect or domain shift to manage.

Status: the pipeline is implemented but the HSV bounds are still placeholders. Calibrating them against the real pieces under real lighting is the next perception task — they are exposed as ROS parameters specifically so that can be done without touching code.

Typed interfaces replace shared memory

In ROS 1, `robotControl` reached directly into an `ARTagListener` object and shared a mutable dict behind a lock. In ROS 2 every boundary is a compiled message contract — which means each node can be tested, replaced or restarted on its own.

INTERFACE	CARRIES	DIRECTION
<code>ExecuteMove.action</code>	<code>game_id</code> , <code>move_string</code> , <code>current_position</code> , <code>new_position</code>	<code>game_manager</code> → <code>robot_control</code>
<code>MoveArm.action</code>	<code>x</code> , <code>y</code> , <code>z</code> in metres, base frame	<code>robot_control</code> → <code>arm_controller</code>
<code>MoveGripper.action</code>	<code>open</code> (bool)	<code>robot_control</code> → <code>gripper_controller</code>
<code>GameState.msg</code>	<code>position</code> , <code>available_moves</code> , <code>is_robot_turn</code> , <code>game_over</code>	<code>game_manager</code> → any subscriber
<code>/vision/board_state</code>	occupancy string, one char per cell	<code>vision_node</code> → <code>robot_control</code>

Why actions, not services or topics

Motion is long-running and can fail. Actions give the caller three things a topic cannot: a goal acknowledgement, streaming feedback, and a terminal result. A failed plan now propagates all the way back to the game loop instead of vanishing.

The async requirement

Calling `spin_until_future_complete` inside an action callback deadlocks under a `MultiThreadedExecutor` — the node is already being spun. The motion sequencer is therefore `async` throughout and `await`s each goal, yielding the executor while it waits.

Game-agnostic by construction

The ROS 1 AR tracker was hardcoded for a single game. Adding a game here means one entry in `centers.py` and one in the `_GAME_TYPES` map — no node code changes.

WHAT EXISTS TODAY

Seven nodes, ~1,170 lines

FILE	LINES	RESPONSIBILITY	REPLACES / NOTES
<code>game_manager.py</code>	246	Lifecycle node · UWAPI game loop	<code>main.py</code> — adds configure/activate states and greedy move selection in a daemon thread
<code>robot_control.py</code>	289	ExecuteMove server · Type taxonomy	<code>robotControl.py</code> — same Type1–Type11 model, now async and driven by action clients
<code>arm_controller.py</code>	112	Movelt 2 planning · OMPL	<code>low_level_controller.py</code> — the LIN → OMPL fix lives here
<code>gripper_controller.py</code>	74	Gripper action server	Replaces the 7x-publish loop with one RELIABLE-QoS publish
<code>vision_node.py</code>	170	ArUco → homography → HSV	<code>ar_track_alvar</code> , <code>findPiece.py</code> , <code>artag_listener.py</code> , <code>transformBroadcast.py</code>
<code>centers.py</code>	89	Board coordinate tables	Ported and made game-agnostic; no ROS dependency
<code>joint_state_bridge.py</code>	55	degrees → radians for Movelt 2	Direct port of <code>joint_states.py</code>
<code>gamesmanros.launch.py</code>	103	Bringup, params, vision toggle	New — <code>vision:=true</code> , <code>game_id:=...</code> , HSV bounds as parameters

Comparable in size to the ROS 1 system (~940 lines) — the gain is not less code, it is that no file can block, corrupt or crash another.

Written and reviewed — not yet validated on hardware

COMPLETE

- All seven nodes implemented end to end
- Four custom interfaces defined and wired
- LIN → OMPL planner migration
- Gripper action server with RELIABLE QoS
- Lifecycle state machine on `game_manager`
- Full OpenCV perception pipeline
- Async refactor — deadlock path removed
- `game_id` moved onto the goal message

BLOCKING HARDWARE TEST

- MoveIt 2 config for the MyCobot 280 — URDF, SRDF, `arm_group`, `joint6_flange`
- Raspberry Pi reflash: Ubuntu 20.04 → 22.04 arm64 for Humble
- Launch ordering — `arm_controller` can currently start before `/joint_states` exists
- HSV bounds are untuned placeholders
- `_pickup` table is empty, so Type1 games have no source stack

NEXT

- Stand up MoveIt 2 config and drive `MoveArm` goals directly from the CLI
- Calibrate HSV against the real pieces under real lighting
- Close the perception loop — detect a human's move from `/vision/board_state`
- Mock UWAPI responses so the motion stack can be tested while the solver API is down

UWAPI is currently unavailable, but only `game_manager` depends on it — the entire motion and perception stack can be exercised without it.

What actually changed

DIMENSION	ROS 1 (ORIGINAL)	ROS 2 (REWRITE)
Control flow	One synchronous Python call stack	Seven independent nodes over typed interfaces
Motion planning	PILZ LIN — Cartesian, fails without a straight IK-feasible path	OMPL RRTConnect — joint-space, IK required only at endpoints
Failure signalling	<code>plan()</code> returns <code>False</code> ; caller ignores it	Action aborts propagate to the game loop and are logged
Gripper	7+ publishes with <code>sleep()</code> , no confirmation	One RELIABLE publish, action result confirms actuation
Recovery	Kill and restart the whole process	Lifecycle transitions; per-node restart
Perception	<code>ar_track_alvar</code> → TF tree → shared dict behind a lock	ArUco homography → HSV → published board-state topic
Camera setup	Intrinsics file + extrinsic TF calibration	None — four corner markers fully determine the warp
Adding a game	AR tracking hardcoded for one game	Two table entries, no node changes
Concurrency	<code>threading.Thread(target=rospy.spin)</code>	<code>MultiThreadedExecutor</code> with async callbacks

The through-line: every failure in the original system traced back to the same decision — treating a distributed robotics problem as a single-threaded script. The rewrite does not patch those failures individually; it removes the structure that produced them.